분산시스템 과제 #9

박종현, 2025-05-23
공과대학 컴퓨터정보통신공학과

과제 목표

1. 4-02 자료 : 16 18쪽
2. 4-03 자료 : 3 15쪽

[ch 4-1]

```
hduser@dist:~/data$ spark shell
Command 'spark' not found, did you mean:
  command 'nspark' from deb nspark (1.7.8B2+git20210317.cb30779-2)
Try: sudo apt install <deb name>
hduser@dist:~/data$ spark-shell
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/05/23 04:39:01 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java cl
asses where applicable
Spark context Web UI available at http://dist:4040
Spark context available as 'sc' (master = local[*], app id = local-1747975144822).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.5.5
      /_/

Using Scala version 2.12.18 (OpenJDK 64-Bit Server VM, Java 1.8.0_452)
Type in expressions to have them evaluated.
Type :help for more information.

scala> val flightData = spark.read.option("inferSchema", "true").option("header", "true").csv("./2015-summary.csv")
flightData: org.apache.spark.sql.DataFrame = [DEST_COUNTRY_NAME: string, ORIGIN_COUNTRY_NAME: string ... 1 more field]

scala> flightData.take(3)
res0: Array[org.apache.spark.sql.Row] = Array([United States,Romania,15], [United States,Croatia,1], [United States,Irel
and,344])

scala>
```

```
scala> flightData.sort("count").take(3)
res1: Array[org.apache.spark.sql.Row] = Array([Moldova,United States,1], [United States,Croatia,1], [United States,Singa
pore,1])

scala> flightData.orderBy(desc("count")).take(3)
res2: Array[org.apache.spark.sql.Row] = Array([United States,United States,370002], [United States,Canada,8483], [Canada
,United States,8399])

scala> flightData.sort("count").explain()
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [count#19 ASC NULLS FIRST], true, 0
   +- Exchange rangepartitioning(count#19 ASC NULLS FIRST, 200), ENSURE_REQUIREMENTS, [plan_id=45]
      +- FileScan csv [DEST_COUNTRY_NAME#17,ORIGIN_COUNTRY_NAME#18,count#19] Batched: false, DataFilters: [], Format: CS
V, Location: InMemoryFileIndex(1 paths)[file:/home/hduser/data/2015-summary.csv], PartitionFilters: [], PushedFilters: [
], ReadSchema: struct<DEST_COUNTRY_NAME:string,ORIGIN_COUNTRY_NAME:string,count:int>



scala> flightData.orderBy(desc("count")).explain()
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [count#19 DESC NULLS LAST], true, 0
   +- Exchange rangepartitioning(count#19 DESC NULLS LAST, 200), ENSURE_REQUIREMENTS, [plan_id=55]
      +- FileScan csv [DEST_COUNTRY_NAME#17,ORIGIN_COUNTRY_NAME#18,count#19] Batched: false, DataFilters: [], Format: CS
V, Location: InMemoryFileIndex(1 paths)[file:/home/hduser/data/2015-summary.csv], PartitionFilters: [], PushedFilters: [
], ReadSchema: struct<DEST_COUNTRY_NAME:string,ORIGIN_COUNTRY_NAME:string,count:int>



scala> Z
```

[ch 4-2]

```
25/05/23 04:46:05 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java cl
asses where applicable
Spark context Web UI available at http://dist:4040
Spark context available as 'sc' (master = local[*], app id = local-1747975568079).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.5.5
      /_/

Using Scala version 2.12.18 (OpenJDK 64-Bit Server VM, Java 1.8.0_452)
Type in expressions to have them evaluated.
Type :help for more information.

scala> val df = spark.read.format("json").load("./2015-summary.json")
df: org.apache.spark.sql.DataFrame = [DEST_COUNTRY_NAME: string, ORIGIN_COUNTRY_NAME: string ... 1 more field]

scala> df.printSchema()
root
 |-- DEST_COUNTRY_NAME: string (nullable = true)
 |-- ORIGIN_COUNTRY_NAME: string (nullable = true)
 |-- count: long (nullable = true)


scala> df.schema
res1: org.apache.spark.sql.types.StructType = StructType(StructField(DEST_COUNTRY_NAME,StringType,true),StructField(ORIG
IN_COUNTRY_NAME,StringType,true),StructField(count,LongType,true))

scala>
```

```
Type :help for more information.

scala> val df = spark.read.format("json").load("./2015-summary.json")
df: org.apache.spark.sql.DataFrame = [DEST_COUNTRY_NAME: string, ORIGIN_COUNTRY_NAME: string ... 1 more field]

scala> df.printSchema()
root
 |-- DEST_COUNTRY_NAME: string (nullable = true)
 |-- ORIGIN_COUNTRY_NAME: string (nullable = true)
 |-- count: long (nullable = true)


scala> df.schema
res1: org.apache.spark.sql.types.StructType = StructType(StructField(DEST_COUNTRY_NAME,StringType,true),StructField(ORIG
IN_COUNTRY_NAME,StringType,true),StructField(count,LongType,true))

scala> df.columns
res2: Array[String] = Array(DEST_COUNTRY_NAME, ORIGIN_COUNTRY_NAME, count)

scala> df.first()
res3: org.apache.spark.sql.Row = [United States,Romania,15]

scala> df.collect()
res4: Array[org.apache.spark.sql.Row] = Array([United States,Romania,15], [United States,Croatia,1], [United States,Irel
and,344], [Egypt,United States,15], [United States,India,62], [United States,Singapore,1], [United States,Grenada,62], [
Costa Rica,United States,588], [Senegal,United States,40], [Moldova,United States,1], [United States,Sint Maarten,325],
[United States,Marshall Islands,39], [Guyana,United States,64], [Malta,United States,1], [Anguilla,United States,41], [B
olivia,United States,30], [United States,Paraguay,6], [Algeria,United States,4], [Turks and Caicos Islands,United States
,230], [United States,Gibraltar,1], [Saint Vincent and the Grenadines,United States,1], [Italy,United States,382], [Unit
ed States,Federated States of Micronesia,69], [Unit...
```

```
scala> df.select("DEST_COUNTRY_NAME").show(2)
+-----------------+
|DEST_COUNTRY_NAME|
+-----------------+
|    United States|
|    United States|
+-----------------+
only showing top 2 rows


scala> df.select("DEST_COUNTRY_NAME", "ORIGIN_COUNTRY_NAME").show(2)
+-----------------+-------------------+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|
+-----------------+-------------------+
|    United States|            Romania|
|    United States|            Croatia|
+-----------------+-------------------+
only showing top 2 rows


scala> df.select(df.col("DEST_COUNTRY_NAME"), "DEST_COUNTRY_NAME")
<console>:24: error: overloaded method value select with alternatives:
  [U1, U2](c1: org.apache.spark.sql.TypedColumn[org.apache.spark.sql.Row,U1], c2: org.apache.spark.sql.TypedColumn[org.a
pache.spark.sql.Row,U2])org.apache.spark.sql.Dataset[(U1, U2)] <and>
  (col: String,cols: String*)org.apache.spark.sql.DataFrame <and>
  (cols: org.apache.spark.sql.Column*)org.apache.spark.sql.DataFrame
 cannot be applied to (org.apache.spark.sql.Column, String)
       df.select(df.col("DEST_COUNTRY_NAME"), "DEST_COUNTRY_NAME")
          ^

scala>
```

```
scala>

scala>

scala>

scala>

scala> df.select(df.col("DEST_COUNTRY_NAME"), col("DEST_COUNTRY_NAME"), 'DEST_COUNTRY_NAME', $"DEST_COUNTRY_NAME").show(
2)
<console>:1: error: unclosed character literal (or use " for string literal "DEST_COUNTRY_NAME")
       df.select(df.col("DEST_COUNTRY_NAME"), col("DEST_COUNTRY_NAME"), 'DEST_COUNTRY_NAME', $"DEST_COUNTRY_NAME").show(
2)
                                                                                                                  ^

scala> df.select(df.col("DEST_COUNTRY_NAME"),col("DEST_COUNTRY_NAME"),'DEST_COUNTRY_NAME',$"DEST_COUNTRY_NAME").show(2)
<console>:1: error: unclosed character literal (or use " for string literal "DEST_COUNTRY_NAME")
       df.select(df.col("DEST_COUNTRY_NAME"),col("DEST_COUNTRY_NAME"),'DEST_COUNTRY_NAME',$"DEST_COUNTRY_NAME").show(2)
                                                                                                                ^

scala> df.select(df.col("DEST_COUNTRY_NAME"),col("DEST_COUNTRY_NAME"),'DEST_COUNTRY_NAME,$"DEST_COUNTRY_NAME").show(2)
+-----------------+-----------------+-----------------+-----------------+
|DEST_COUNTRY_NAME|DEST_COUNTRY_NAME|DEST_COUNTRY_NAME|DEST_COUNTRY_NAME|
+-----------------+-----------------+-----------------+-----------------+
|    United States|    United States|    United States|    United States|
|    United States|    United States|    United States|    United States|
+-----------------+-----------------+-----------------+-----------------+
only showing top 2 rows


scala>
```

```
Type :help for more information.

scala> df.select(expr("DEST_COUNTRY_NAME AS destination")).show(2)
<console>:23: error: not found: value df
       df.select(expr("DEST_COUNTRY_NAME AS destination")).show(2)
       ^

scala> val df = spark.read.format("json").load("./2015-summary.json")
df: org.apache.spark.sql.DataFrame = [DEST_COUNTRY_NAME: string, ORIGIN_COUNTRY_NAME: string ... 1 more field]

scala> df.select(expr("DEST_COUNTRY_NAME AS destination")).show(2)
+-------------+
|  destination|
+-------------+
|United States|
|United States|
+-------------+
only showing top 2 rows


scala> df.selectExpr("*", "(DEST_COUNTRY_NAME = ORIGIN_COUNTRY_NAME) as withinCountry").show(2)
+-----------------+-------------------+-----+-------------+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|withinCountry|
+-----------------+-------------------+-----+-------------+
|    United States|            Romania|   15|        false|
|    United States|            Croatia|    1|        false|
+-----------------+-------------------+-----+-------------+
only showing top 2 rows


scala>
```

```
only showing top 2 rows


scala> df.selectExpr("*", "(DEST_COUNTRY_NAME = ORIGIN_COUNTRY_NAME) as withinCountry").show(2)
+-----------------+-------------------+-----+-------------+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|withinCountry|
+-----------------+-------------------+-----+-------------+
|    United States|            Romania|   15|        false|
|    United States|            Croatia|    1|        false|
+-----------------+-------------------+-----+-------------+
only showing top 2 rows


scala> val df1 = df.selectExpr("*", "(DEST_COUNTRY_NAME = ORIGIN_COUNTRY_NAME) as withinCountry")
df1: org.apache.spark.sql.DataFrame = [DEST_COUNTRY_NAME: string, ORIGIN_COUNTRY_NAME: string ... 2 more fields]

scala> df1.schema
res3: org.apache.spark.sql.types.StructType = StructType(StructField(DEST_COUNTRY_NAME,StringType,true),StructField(ORIG
IN_COUNTRY_NAME,StringType,true),StructField(count,LongType,true),StructField(withinCountry,BooleanType,true))

scala> df1.show(2)
+-----------------+-------------------+-----+-------------+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|withinCountry|
+-----------------+-------------------+-----+-------------+
|    United States|            Romania|   15|        false|
|    United States|            Croatia|    1|        false|
+-----------------+-------------------+-----+-------------+
only showing top 2 rows


scala>
```

```
scala> df.withColumn("numberOne", lit(1)).show(2)
+-----------------+-------------------+-----+---------+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|numberOne|
+-----------------+-------------------+-----+---------+
|    United States|            Romania|   15|        1|
|    United States|            Croatia|    1|        1|
+-----------------+-------------------+-----+---------+
only showing top 2 rows


scala> df.withColumn("withinCountry", expr("ORIGIN_COUNTRY_NAME == DEST_COUNTRY_NAME")).show(2)
+-----------------+-------------------+-----+-------------+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|withinCountry|
+-----------------+-------------------+-----+-------------+
|    United States|            Romania|   15|        false|
|    United States|            Croatia|    1|        false|
+-----------------+-------------------+-----+-------------+
only showing top 2 rows


scala> df.withColumnRenamed("DEST_COUNTRY_NAME", "dest").columns
res7: Array[String] = Array(dest, ORIGIN_COUNTRY_NAME, count)

scala> df.drop("ORIGIN_COUNTRY_NAME").columns
res8: Array[String] = Array(DEST_COUNTRY_NAME, count)

scala> df.withColumn("count2", col("count").cast("string"))
res9: org.apache.spark.sql.DataFrame = [DEST_COUNTRY_NAME: string, ORIGIN_COUNTRY_NAME: string ... 2 more fields]

scala> |
```

```
scala> df.filter(col("count") < 2 ).show(2)
+-----------------+-------------------+-----+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|
+-----------------+-------------------+-----+
|    United States|            Croatia|    1|
|    United States|          Singapore|    1|
+-----------------+-------------------+-----+
only showing top 2 rows


scala> df.where ("count < 2").show(2)
+-----------------+-------------------+-----+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|
+-----------------+-------------------+-----+
|    United States|            Croatia|    1|
|    United States|          Singapore|    1|
+-----------------+-------------------+-----+
only showing top 2 rows


scala> df.where(col("count") < 2).where(col("ORIGIN_COUNTRY_NAME") =!= "Croatia").show(2)
+-----------------+-------------------+-----+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|
+-----------------+-------------------+-----+
|    United States|          Singapore|    1|
|          Moldova|      United States|    1|
+-----------------+-------------------+-----+
only showing top 2 rows


scala> |
```


```
+-----------------+-------------------+-----+
|    United States|            Croatia|    1|
|    United States|          Singapore|    1|
+-----------------+-------------------+-----+
only showing top 2 rows


scala> df.where ("count < 2").show(2)
+-----------------+-------------------+-----+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|
+-----------------+-------------------+-----+
|    United States|            Croatia|    1|
|    United States|          Singapore|    1|
+-----------------+-------------------+-----+
only showing top 2 rows


scala> df.where(col("count") < 2).where(col("ORIGIN_COUNTRY_NAME") =!= "Croatia").show(2)
+-----------------+-------------------+-----+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|
+-----------------+-------------------+-----+
|    United States|          Singapore|    1|
|          Moldova|      United States|    1|
+-----------------+-------------------+-----+
only showing top 2 rows
```

```
scala> df.select("ORIGIN_COUNTRY_NAME", "DEST_COUNTRY_NAME").distinct().count()
res16: Long = 256

scala> |
```

```
scala> val seed = 5
seed: Int = 5

scala> val withReplacement = false
withReplacement: Boolean = false

scala> val fraction = 0.5
fraction: Double = 0.5

scala> df.sample(withReplacement, fraction, seed).count()
res17: Long = 138

scala> seed = 100
<console>:23: error: reassignment to val
       seed = 100
            ^

scala> df.sample(withReplacement, fraction, seed).count()
res18: Long = 138

scala> df.sample(withReplacement, fraction, seed).show(5)
+-----------------+-------------------+-----+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|
+-----------------+-------------------+-----+
|    United States|            Romania|   15|
|    United States|            Croatia|    1|
|    United States|              India|   62|
|    United States|          Singapore|    1|
|    United States|            Grenada|   62|
+-----------------+-------------------+-----+
only showing top 5 rows
```

```
scala> val dataFrames = df.randomSplit(Array(0.25, 0.75), seed)
dataFrames: Array[org.apache.spark.sql.Dataset[org.apache.spark.sql.Row]] = Array([DEST_COUNTRY_NAME: string, ORIGIN_COU
NTRY_NAME: string ... 1 more field], [DEST_COUNTRY_NAME: string, ORIGIN_COUNTRY_NAME: string ... 1 more field])

scala> dataFrames(0).count()
res20: Long = 71

scala> dataFrames(1).count()
res21: Long = 185

scala> df.count()
res22: Long = 256

scala> dataFrames = df.randomSplit(Array(0.1, 0.2, 0.7), seed)
<console>:25: error: reassignment to val
       dataFrames = df.randomSplit(Array(0.1, 0.2, 0.7), seed)
                  ^

scala> val dataFrames = df.randomSplit(Array(0.1, 0.2, 0.7), seed)
dataFrames: Array[org.apache.spark.sql.Dataset[org.apache.spark.sql.Row]] = Array([DEST_COUNTRY_NAME: string, ORIGIN_COU
NTRY_NAME: string ... 1 more field], [DEST_COUNTRY_NAME: string, ORIGIN_COUNTRY_NAME: string ... 1 more field], [DEST_CO
UNTRY_NAME: string, ORIGIN_COUNTRY_NAME: string ... 1 more field])

scala> dataFrames(0).count()
res23: Long = 34

scala> dataFrames(1).count()
res24: Long = 52

scala> dataFrames(2).count()
res25: Long = 170
```

```
scala> dataFrames(2).count()
res25: Long = 170

scala> df.sort("count").show(5)
+--------------------+-------------------+-----+
|   DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|
+--------------------+-------------------+-----+
|               Malta|      United States|    1|
|Saint Vincent and...|      United States|    1|
|       United States|            Croatia|    1|
|       United States|          Gibraltar|    1|
|       United States|          Singapore|    1|
+--------------------+-------------------+-----+
only showing top 5 rows


scala> df.orderBy("count", "DEST_COUNTRY_NAME").show(5)
+-----------------+-------------------+-----+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME|count|
+-----------------+-------------------+-----+
|     Burkina Faso|      United States|    1|
|    Cote d'Ivoire|      United States|    1|
|           Cyprus|      United States|    1|
|         Djibouti|      United States|    1|
|        Indonesia|      United States|    1|
+-----------------+-------------------+-----+
only showing top 5 rows


scala>
```

```
+-----------------+-------------------+------+
only showing top 5 rows


scala> df.orderBy(expr("count").desc).show(5)
+-----------------+-------------------+------+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME| count|
+-----------------+-------------------+------+
|    United States|      United States|370002|
|    United States|             Canada|  8483|
|           Canada|      United States|  8399|
|    United States|             Mexico|  7187|
|           Mexico|      United States|  7140|
+-----------------+-------------------+------+
only showing top 5 rows


scala> df.orderBy(desc("count"), asc("DEST_COUNTRY_NAME")).show(5)
+-----------------+-------------------+------+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME| count|
+-----------------+-------------------+------+
|    United States|      United States|370002|
|    United States|             Canada|  8483|
|           Canada|      United States|  8399|
|    United States|             Mexico|  7187|
|           Mexico|      United States|  7140|
+-----------------+-------------------+------+
only showing top 5 rows


scala>
```

```
|    United States|             Mexico|  7187|
|           Mexico|      United States|  7140|
+-----------------+-------------------+------+
only showing top 5 rows


scala> df.orderBy(desc("count"), asc("DEST_COUNTRY_NAME")).show(5)
+-----------------+-------------------+------+
|DEST_COUNTRY_NAME|ORIGIN_COUNTRY_NAME| count|
+-----------------+-------------------+------+
|    United States|      United States|370002|
|    United States|             Canada|  8483|
|           Canada|      United States|  8399|
|    United States|             Mexico|  7187|
|           Mexico|      United States|  7140|
+-----------------+-------------------+------+
only showing top 5 rows


scala> df.rdd.getNumPartitions
res30: Int = 1

scala> df.repartition(5)
res31: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [DEST_COUNTRY_NAME: string, ORIGIN_COUNTRY_NAME: string
... 1 more field]

scala> df.repartition(5, col("DEST_COUNTRY_NAME"))
res32: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [DEST_COUNTRY_NAME: string, ORIGIN_COUNTRY_NAME: string
... 1 more field]

scala>
```